

单测框架设计

目标理解：为新IR设计单测框架，能够将各部分串起来运行测试。分析旧IR和静态图的测试逻辑，并写出类似的框架用于新IR。

一、设计方案

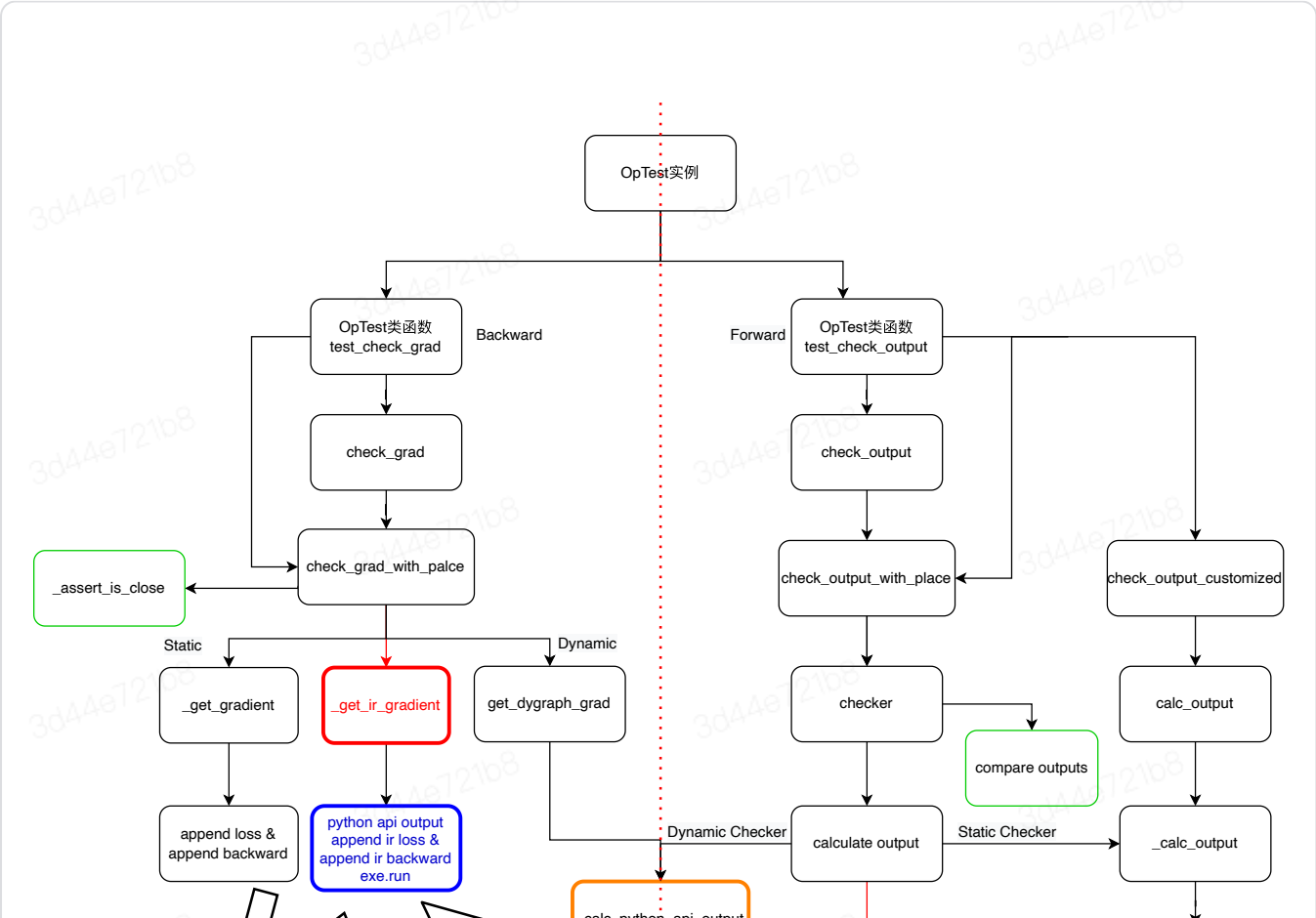
推荐路线：参考动态图测试，复用其python_api的使用逻辑。

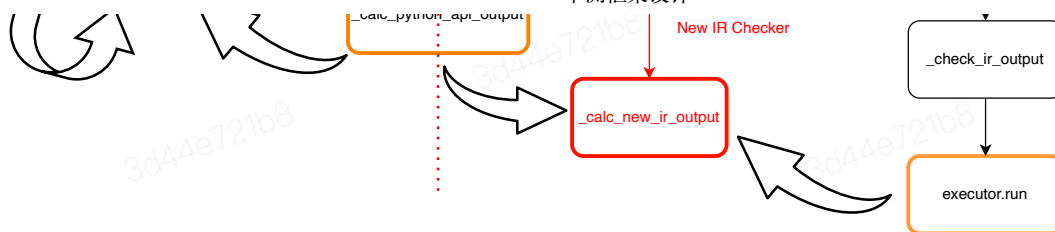
具体设计：

前向：仿照动态图调用python_api的逻辑，调用日升后续写好的接口（封装在python_api中），基本逻辑一致，需要考虑传参以及函数放置位置、校验对比逻辑放在哪里。调用接口完成组网，拿到输出，还需要按照旧IR逻辑，放入执行器中进行run。

反向：调用儒婷的backward接口，逻辑也与动态图类似。

梳理OpTest的流程图，以及新加设计。





前向：增加核心函数`_calc_new_ir_output`。

- 位置：放在新的checker下，用来对比new ir与其他的输出。也可以被其他checker复用。
- 功能：
 - 仿照动态图的函数`_calc_python_api_output`调用日升封装好的`python_api`进行组网。
 - 仿照静态图部分调用新IR的逻辑放入执行器中run。
- 设计考虑：
 - 通过`python_api`调用，逻辑与动态图比较接近；单独写一个函数可以减少对原代码的影响。
 - 需要在接口里新加一个控制`check_new_ir`的flag来控制单测是否跑新IR，使用日升的flag来控制当前组网运行的环境切换到新IR。

反向：增加核心函数`_get_ir_gradient`。

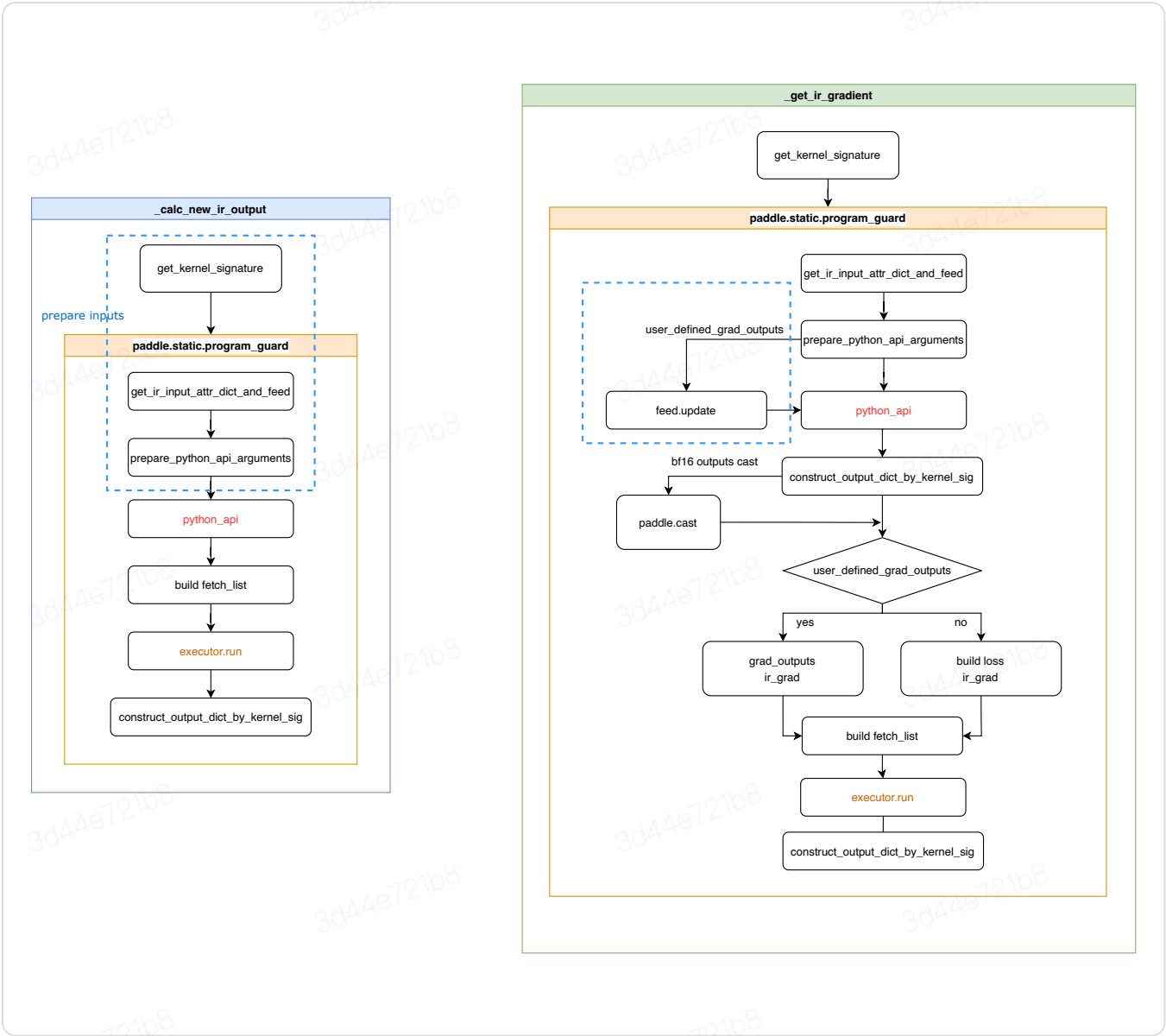
- 位置：与静态图动态图的`grad`函数放在一起，由`check_grad`中选择调用。
- 功能：与静态图的`_get_gradient`基本类似，放入儒婷的`backward`接口，loss部分的处理需要修改。
- 需要结合动态图的`python_api`组网以及静态图的`append loss/backward`逻辑。

二、代码实现

1. 前向组网，新建New IR Checker，直接在`check_output_with_place`里调用`check()`

- `_calc_new_ir_output`:
 - 准备参数
 - 将dygraph部分获取`kernel_sig`的逻辑整理成函数`get_kernel_signature`复用。
 - 将`PrimForwardChecker`中的类函数整理成函数`get_ir_input_attr_dict_and_feed`用来获取`input`、`attr`、`feed`等参数
 - 调用`OpTestUtils.prepare_python_api_arguments`函数进行参数args的准备
 - `fetchlist`不含name
 - `return numpy`设置为true/不设置默认为true
 - 调用`python_api`，将返回结果输送给`fetchlist`。
 - `executor.run`，`program`调用new ir的默认`program`
 - 为`static.program_guard`添加`new_ir guard`，保证flag在新旧IR之间切换时正确设置。

- 使用construct_output_dict_by_kernel_sig构建返回result，将结果与name对应，返回dict。



2. 反向梯度，新增函数

- `_get_ir_gradient`:
 - 先进行前向组网，调用`_calc_new_ir_output`，获得前向输出、输入。
 - 对有无user defined grad分类讨论

`</> user_defined_grad_outputs不为空`

Python | 收起 ^

```
1 if user_defined_grad_outputs is not None:
2     # user_defined_grad_outputs here are numpy
    arrays
3     if not isinstance(user_defined_grad_outputs,
        list):
4         user_defined_grad_outputs =
        [user_defined_grad_outputs]
5         for grad_out_value, idx in zip(
6             user_defined_grad_outputs,
```

```
7         range(len(user_defined_grad_outputs)),
8     ):
9         grad_val = paddle.static.data(
10             # 使用data构建数据
11             name='val_grad_%s' % idx,
12             shape=grad_out_value.shape,
13             dtype=grad_out_value.dtype,
14         )
15         grad_outputs.append(grad_val)
16         feed.update({'val_grad_%s' % idx:
17                     grad_out_value})
18         # delete the inputs which no need to
19         # calculate grad
20         for no_grad_val in no_grad_set:
21             del static_inputs[no_grad_val]
```

- 在执行器侧，需要将data和fetch两个op放在组网最前端，因此对于user_defined_grad_outputs的数据构建也需要放在调用python api之前。
 - 调用前向python api
 - 构建outputs和fetchlist
 - 针对bfloat16类型进行cast
 - 根据有无user_defined_grad_outputs进行反向组网
 - executor.run，获得结果。